LAB 5: STATIC CODE ANALYSIS

Name: Uthkarsha
SRN: PES1UG24AM413
Section: AIML G

1. Objective

To improve the quality, security, and style of the assigned Flight Reservation System using static analysis tools. Pylint checks code quality, Bandit identifies common security risks, and Flake8 checks style and formatting. The program was reviewed and updated to address unsafe patterns and maintainability issues.

2. Program and Tools

The assigned program manages flight seats, cancellations, baggage surcharges, passenger identifiers, boarding PINs, and flight-manifest export.

Tool	Purpose
Pylint	Checks code quality, naming, documentation, and programming practices.
Bandit	Checks common insecure Python coding patterns.
Flake8	Checks style and formatting, including whitespace and line length.

3. Issue Documentation and Fixes

The line numbers below refer to the original file and may differ in the corrected version. This table is based on source-code inspection; confirm exact findings and line numbers against the generated tool reports.

Issue	Type	Original line(s)	Description / Fix
Unsafe eval()	Security	45–48	Expression evaluation could run arbitrary Python. Replaced with a validated numeric surcharge rate.
Shell command with filename	Security	59–63	Filename was concatenated into os.system(). Removed shell invocation.
Hard-coded secret	Security	75, 84	Token was embedded and printed. Removed it from the program.
Non-secure PIN source	Security	55–57	random was used for a PIN. Replaced with secrets.randbelow().
MD5 digest	Security	51–52	Replaced MD5 with SHA-256.
Mutable default list	Bug	15	Changed baggage_tags=[] to None and initialized a new list inside the function.
Bare except	Reliability	27–32	Replaced broad exception suppression with explicit lookup and a return value.
Unsafe file handling	Quality	59–62	Used a context manager and UTF-8 encoding.
Unused import / variable	Style	4, 36	Removed math and airport_fee.
Naming / documentation	Style	Multiple	Renamed functions to snake_case and added docstrings and a main guard.

4. Before-and-After Results

Tool	Before	After	Status
Pylint	Not run in this preparation environment	Not run in this preparation environment	Run locally and record actual scores
Bandit	Not run in this preparation environment	Not run in this preparation environment	Run locally and review findings
Flake8	Not run in this preparation environment	Not run in this preparation environment	Run locally and review output

Verification note: The corrected program was syntax-compiled and smoke-tested. Pylint, Bandit, and Flake8 could not be installed in the preparation environment because package-index access was unavailable. Actual scores and reports must be generated locally before final submission.

5. Reflection

5.1 What is static code analysis?
It examines source code without executing it to identify possible bugs, security risks, and style violations.

5.2 What are the purposes of Pylint, Bandit, and Flake8?
Pylint focuses on code quality and programming practices, Bandit on common security risks, and Flake8 on Python style and formatting.

5.3 What improvements were made?
Unsafe eval and shell use were removed; the hard-coded token was removed; PIN generation and hashing were updated; the mutable default and file handling were corrected; and naming, documentation, and validation were improved.

5.4 Why is static analysis useful?
It catches issues earlier, improves readability and maintainability, and provides repeatable checks. The tools complement one another, so using all three gives broader coverage.